1. Matriz de Code Smells e Violações Arquiteturais

#	Trecho / Linha do Código Afetado	Code Smell ou Princípio Violado	Juros Decorrentes (Impacto Prático)
1	`double basePrice, double calculatedPrice` (Linhas 15, 25, 28, etc.)	**Primitive Obsession**	Uso de tipos primitivos (`double`) para manipular valores financeiros gera erros imprecisos de arredondamento em ponto flutuante, além de espalhar regras de arredondamento pelo código.
2	Assinatura do método `processMonthlyBilling(...)` com 8 parâmetros (Linhas 15-17)	**Long Parameter List**	Dificulta a leitura, a manutenção e a reutilização do método. Aumenta drasticamente a probabilidade de troca acidental de ordem de argumentos do mesmo tipo (ex.: `planType` e `promoCode`).

3	`DriverManager.getConnection(...)` e `new SimpleEmail()` (Linhas 61 e 90)	**Dependência Direta de I/O / Violação do SRP (Single Responsibility Principle)**	O código de negócio está rigidamente acoplado à infraestrutura de banco de dados e servidor SMTP. Isso impede a execução de testes unitários automatizados em esteiras de integração contínua (CI) sem subir serviços externos.
4	`LocalDate.now()` estático (Linha 21)	**Hardcoded Time Dependency / Acoplamento Temporal**	Torna os cálculos e regras sensíveis a datas (como desconto de *Black Friday* no mês 11) não determinísticos, impedindo a criação de testes de unidade confiáveis que simulem diferentes datas.

5	Método `processMonthlyBilling` acumulando > 80 linhas	**Long Method / God Method**	O método calcula regras de desconto, conecta ao banco de dados, processa cobrança, insere registros no banco e dispara e-mails. Possui altíssima complexidade ciclomática e risco elevado de regressões a cada modificação.
6	Bloco `catch (Exception e)` abrangente (Linha 100)	**Swallowed / Masked Exceptions**	Captura genérica que engole qualquer exceção e limuta a saída a um log em `List<String>`, mascarando erros graves de infraestrutura (ex.: indisponibilidade do banco ou erro de rede no e-mail) e dificultando o diagnóstico em produção.